

Dual-Arm Robotics: Teleoperation, Stereoscopic Vision, and Policy Learning

Madhavan S | Robotics R&D Internship, 2026

Abstract

During this internship I worked on three connected pieces of a dual-arm manipulation pipeline, using the AgileX Piper platform. The first was a real-time bimanual teleoperation system built around a Meta Quest 3 headset, where hand movements are streamed over WebSocket and turned into joint commands using Pinocchio and Pink inverse kinematics. The second was a stereoscopic vision pipeline that streams live ZED Mini stereo footage to the headset at under 50 ms latency, using POSIX shared memory for fast frame transfer and WebXR layer based rendering so each eye gets true depth. The third was training and evaluating an ACT (Action Chunking with Transformers) policy on a dual bottle pick task, where the robot picked up both bottles and placed them at the goal positions in simulation.

1. Introduction

Dual-arm manipulation means coordinating two robot arms in a shared workspace, while handling collision avoidance, grasp planning, and joint constraints at the same time. Teaching a robot to do this through imitation learning adds another layer on top: demonstrations need to be high quality, repeatable, and quick to collect.

I worked on three projects that together cover different parts of that pipeline. The first is a real-time teleoperation system that lets someone control both AgileX Piper arms with a Meta Quest 3 headset, which is also the main way I captured bimanual demonstration data. The second is a stereo vision pipeline that sends live ZED Mini footage into the headset, so the operator gets real depth perception of the workspace while teleoperating. The third is training and evaluating an ACT policy on a pick dual bottles task inside a simulation benchmark, using demonstrations collected on the Piper platform.

The AgileX Piper is a 6 DOF manipulator. It is lighter and cheaper than higher DOF research arms, though that comes with a more limited workspace. All three projects were built to run entirely on local hardware, with no cloud dependency, to keep the setup practical for a small lab.

2. Real-Time Bimanual Teleoperation via Meta Quest 3

2.1 Why Teleoperation

Hand programming a robot for complex manipulation is slow and does not generalize well. It is faster to just have a person perform the task while the robot records joint trajectories and camera observations, and use those recordings as training data. So teleoperation is not just a convenience here, it is how the training dataset gets built in the first place.

The Meta Quest 3 was a natural choice for input. It is consumer hardware, tracks 6 DOF hand pose well, and is inexpensive relative to dedicated motion capture rigs. The main challenge was getting pose data off

the headset and turned into robot joint commands fast enough that the operator does not feel any lag.

2.2 Architecture

The system runs as three independent OS processes. A main process runs an asyncio WebSocket server that receives JSON pose frames from the Quest 3 and routes them into per-arm queues, and does nothing else. A right-arm process and a left-arm process each own their own Pinocchio model, Pink IK solver, EMA filter, and CAN connection, and run on separate CPU cores so a slow IK solve on one side never holds up the other.

Each queue is capped at two entries. That way, each arm process always picks up the freshest pose available and discards anything older.

2.3 Control Loop

- **Deadband filter:** movement under 5 mm is suppressed so small controller tremors do not turn into robot jitter.
- **Calibration:** the first A+B (or X+Y) button press locks in the current controller pose and end effector position as the reference frame, and computes the rotation between Quest space and robot base space.
- **Coordinate remap:** Quest -Z maps to Robot +X, Quest -X maps to Robot +Y, Quest +Y maps to Robot +Z.
- **IK solve:** Pink's FrameTask targets the remapped SE(3) pose, and OSQP solves the constrained velocity QP at 100 Hz while respecting joint limits.
- **EMA smoothing:** alpha of 0.2 on arm joints and 0.1 on wrist joints, enough to take the edge off without adding noticeable lag.
- **Send:** filtered joint angles go out over CAN bus, and the gripper maps directly from the trigger.

If the CAN hardware is not connected, each process automatically drops into a simulation mode, which was useful during development.

2.4 Technology Stack

Component	Technology
Input device	Meta Quest 3 (WebXR API)
Transport	Secure WebSocket (WSS) over Wi-Fi
Robot hardware	AgileX Piper dual-arm manipulators (CAN bus)
IK solver	Pinocchio + Pink (OSQP backend)
Hardware driver	CAN-based low-level arm control
Runtime	Python 3, asyncio, multiprocessing

2.5 How It Went

Both arms ran independently and responsively inside one application. The motion felt fluid, hand movements translated into end effector motion with no noticeable delay in normal use. One thing the multi-process design handled well was fault isolation: when one arm's IK failed briefly during testing, the other arm kept running without interruption.

3. Stereoscopic Vision Streaming to the Meta Quest 3

3.1 The Problem

Teleoperation without a proper visual feed is guesswork. The operator needs to see what the robot sees, ideally with depth, to judge distances and plan grasps. The obvious answer is to stream stereo camera video from the workspace directly into the operator's headset.

Two things mattered most here. First, latency: above roughly 100 ms the feed starts to feel disconnected and operator performance drops. Second, keeping the setup simple, no native headset apps, no cloud relay, no proprietary capture hardware, just the headset's built-in browser and a local network connection.

3.2 Pipeline

The ZED Mini is a dual-lens stereo camera capturing synchronized left and right frames at 1920x1080 and 30 fps. Depth computation was disabled at capture time since only the raw stereo images were needed for rendering, not a depth map.

Left and right frames are concatenated into a side-by-side composite and written into a named POSIX shared memory block. Shared memory was chosen over a queue or pipe for a simple reason: at 1920x1080 stereo, each composite frame is around 6 MB, and a queue would copy and serialize that on every frame. Shared memory lets the streaming server read the same buffer the capture process wrote to, with no copy at all.

The streaming server reads the shared buffer at 60 Hz, splits it into left and right halves, encodes each as JPEG at quality 95, and pushes the stereo pair over a TLS encrypted WebSocket. TLS is not optional here, WebXR will not start an immersive session without a secure context. Instead of routing through something like ngrok, which added 200 to 544 ms of latency in testing, I used mkcert to generate a locally trusted certificate at launch, bound to the host machine's current local IP, and set it to regenerate automatically if the network interface changes.

3.3 Per-Eye Rendering

On the Quest 3, the browser opens an immersive-vr WebXR session. The left and right frames arrive as a stereo pair and get uploaded as two separate GPU textures. Each is assigned to a different Three.js render layer mask, mask 1 for the left eye and mask 2 for the right, and the WebXR renderer fills each eye's framebuffer from its matching texture.

This is different from naive side-by-side rendering, where both eyes receive the full composite and the headset optics handle the split, an approach that degrades with lens distortion. The layer API guarantees

at the rendering level that each eye only ever sees its own frame, which gives clean binocular disparity matching the ZED Mini’s 120 mm physical baseline.

3.4 Performance

On a local 802.11ac network, latency stayed under 50 ms consistently. Depth perception was stable across object distances with no ghosting or cross-eye bleed. JPEG at quality 95 showed no visible artifacts at normal viewing distance, and dropping to quality 80 roughly halved the per-frame payload with no noticeable loss in image quality.

Table 1: End-to-end stereoscopic streaming pipeline.

Stage	Description
ZED Mini	Captures synchronized left/right frames at 1920x1080, 30 fps. Depth disabled to reduce CPU load.
Capture	SDK retrieves per-eye BGRA frames, alpha dropped, frames concatenated into a side-by-side composite (H x 2 x W).
Shared memory	POSIX shared memory holds the composite, no inter-process copy overhead.
Streaming server	Reads buffer at 60 Hz, encodes each half as JPEG (quality 95), pushes stereo pair over WSS.
Transport	TLS WebSocket over local 802.11ac Wi-Fi, certificate auto-generated via mkcert.
Quest 3	WebXR immersive-vr session, left frame to layer 1 (left eye), right frame to layer 2 (right eye) via Three.js objects.

4. ACT Policy Evaluation on RoboTwin 2.0

4.1 Background

RoboTwin 2.0 is a simulation benchmark built on the SAPIEN physics engine, with 50 dual-arm manipulation tasks and pre-collected expert demonstration datasets. I used it to train and evaluate an ACT policy on the pick dual bottles task with the Piper robot.

4.2 The Task

The pick dual bottles task asks the robot to pick up two bottles from randomized positions on a table and place them at fixed targets, in a pick, lift, and place sequence with both arms working in parallel. The Piper’s 6 DOF kinematics push it toward lateral grasps rather than top-down ones, which makes the pick step a bit trickier. The two arms sit symmetrically with 0.6 m of separation.

4.3 Dataset and Training

I used 50 expert demonstrations from the official RoboTwin 2.0 dataset. Each demo has synchronized observations from three cameras, one head-mounted for a global view and two wrist-mounted for close-up views from each arm, plus joint states and end-effector poses at every timestep. Episodes average 63.5 timesteps. I preprocessed the data into ACT format, pairing joint trajectories and camera streams into observation-action sequences.

Training used a chunk size of 50, batch size 8, learning rate of $1e-5$, KL weight of 10, and up to 6000 epochs on a single GPU. I kept the checkpoint with the lowest validation loss.

4.4 Policy Architecture

ACT predicts 50 future actions at once rather than one at a time, which cuts down on compounding errors during execution. The model has three parts. A visual encoder runs ResNet-18, pretrained on ImageNet, on each of the three 480x640 camera inputs independently, producing 900 tokens of 512 dimensions in total. A transformer encoder combines those visual tokens with the current joint state and a CVAE latent variable into a 902-token context, using 4 layers with 8 attention heads (the latent is set to zero at evaluation time). A transformer decoder attends to the encoder output across 7 layers and outputs a 50x14 action chunk, 6 joint angles plus 1 gripper value per arm. Total parameter count is 83.9 million.

4.5 Evaluation

I ran 100 evaluation episodes with random seeds that did not overlap with the training data, so the policy always faced object configurations it had not trained on. Each episode runs for up to 400 steps. At every step the policy takes in camera images and joint states and outputs a 50-step action chunk, and overlapping predictions are blended with temporal aggregation to smooth execution.

The ACT policy, trained on just 50 demonstrations, successfully picked up both bottles from their randomized positions and moved them to the goal locations. The full pick, lift, place sequence worked end to end on the Piper.

5. How the Projects Connect

The teleoperation system and the stereo vision pipeline were built to work together, the operator wears the Quest 3, gets the stereo depth feed from the ZED Mini, and drives both Piper arms with the controllers. The joint trajectories and camera frames recorded during those sessions become the demonstration data for policy training. The policy evaluation work then takes that kind of data and tests how well a learned policy can reproduce the behavior in simulation.

For the RoboTwin 2.0 evaluation specifically, I used the official pre-collected dataset rather than my own recordings, since that is what the benchmark expects. But the teleoperation and stereo systems already run on physical hardware, so the pipeline for collecting custom Piper demonstrations and training policies on them is in place and ready to use.

Next steps from here would be scaling up the number of demonstrations (the RoboTwin 2.0 dataset goes up to 500 per task), trying stronger policy architectures such as DP3, which uses 3D point clouds, and eventually running trained policies on the real arms instead of just in simulation.

6. Conclusion

Across this project I built a teleoperation system for dual Piper arms using a Quest 3 headset, a stereo vision pipeline for streaming ZED Mini footage into the headset with genuine per-eye depth, and trained and evaluated an ACT policy on the pick dual bottles task, where the Piper successfully picked up both

bottles and reached the goal positions in simulation. Each piece was built to be self-contained and deployable without cloud infrastructure or proprietary middleware, with the whole stack running on a single workstation.

References

- [1] T. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," *Robotics: Science and Systems*, 2023.
- [2] T. Chen et al., "RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation," *arXiv:2506.18088*, 2025.
- [3] Y. Ze et al., "3D Diffusion Policy," *arXiv:2403.03954*, 2024.
- [4] C. Cheng et al., "Open-TeleVision: Teleoperation with Immersive Active Visual Feedback," *arXiv:2407.01512*, 2024.
- [5] R. Frossard, M. Geyer, and S. Bauer, "Low-Latency WebRTC Video Streaming for Robot Teleoperation," *IEEE IROS Workshop on Robot Perception*, 2023.